

---

# **Application Mobile *AfriMarket* Marketplace Agricole P2P pour l'Afrique Centrale**

---

Membre du groupe :

- TEUMOU CABREL
- NDAFEU WENDY
- YANAVA
- FABILLA

## Remerciements

---

Nous adressons en premier lieu nos plus sincères remerciements aux membres du jury qui ont bien voulu consacrer de leur précieux temps à l'évaluation de ce travail. Leurs remarques constructives contribueront à l'amélioration de nos pratiques professionnelles.

Nos remerciements vont ensuite à **M. FOMEKONG** enseignant référent, dont la disponibilité, les orientations pédagogiques et les conseils avisés ont guidé chaque étape de ce projet. Sa rigueur académique nous a permis d'aborder les difficultés techniques avec méthode.

Nous exprimons également notre gratitude à l'ensemble des tuteurs et maîtres de stage qui ont suivi l'évolution de ce travail au quotidien, pour leur patience et la transmission de leur expérience professionnelle.

Enfin, nous remercions les membres de l'administration de l'UIGEE et de Keyce Informatique & IA pour leur soutien institutionnel et les conditions d'étude qu'ils ont su créer.

## Résumé

---

AfriMarket est une application mobile de type marketplace agricole pair-à-pair conçue pour le contexte camerounais et, plus largement, pour l'Afrique centrale francophone. Elle met en relation directe des producteurs agricoles avec des consommateurs urbains, en s'appuyant sur un réseau de points relais pour la logistique de livraison et sur le Mobile Money pour les paiements. Le projet comprend trois composantes logicielles : une application *consommateur* (AfriMarket Client), une application *producteur* (AfriMarket Producteur) et une interface d'administration web, toutes alimentées par un serveur backend RESTful.

Ce rapport décrit le processus complet qui a conduit de l'identification du problème de distribution agricole en milieu urbain jusqu'à la livraison d'une solution fonctionnelle. Après une analyse de l'état de l'art des plateformes agri-tech existantes, nous exposons les choix de conception, l'architecture logicielle retenue et la mise en œuvre technique, avant de présenter les résultats obtenus et les perspectives d'amélioration.

**Mots-clés :** application mobile, marketplace agricole, React Native, Spring Boot, Mobile Money, Cameroun, pair-à-pair, logistique, [Representational State Transfer \(REST\)](#).

## Abstract

---

AfriMarket is a peer-to-peer agricultural marketplace mobile application designed for the Cameroonian context and, more broadly, for French-speaking Central Africa. It directly connects agricultural producers with urban consumers through a relay-point delivery network and Mobile Money payments. The project consists of three software components : a *consumer* application (AfriMarket Client), a *producer* application (AfriMarket Producteur), and a web-based administration interface, all powered by a RESTful backend server.

This report describes the complete process from the identification of the agricultural distribution problem in urban areas to the delivery of a functional solution. After an analysis of the state of the art in existing agri-tech platforms, we present the design choices, software architecture, and technical implementation, before discussing results and future improvements.

**Keywords :** mobile application, agricultural marketplace, React Native, Spring Boot, Mobile Money, Cameroon, peer-to-peer, logistics, REST.

## Table des figures

---

|     |                                                                      |    |
|-----|----------------------------------------------------------------------|----|
| 3.1 | Diagramme entité-association simplifié d'AfriMarket .....            | 11 |
| 3.2 | Diagramme des cas d'utilisation d'AfriMarket.....                    | 11 |
| 3.3 | Cycle de vie d'une offre dans AfriMarket.....                        | 12 |
| 3.4 | Architecture en couches d'AfriMarket.....                            | 13 |
| 4.1 | Maquette de l'écran « Découvrir » de l'application consommateur..... | 16 |

## Liste des tableaux

---

|     |                                                            |    |
|-----|------------------------------------------------------------|----|
| 2.1 | Comparaison des plateformes agri-tech existantes . . . . . | 6  |
| 3.1 | Matrice des besoins fonctionnels par acteur . . . . .      | 9  |
| 3.2 | Besoins non fonctionnels du système AfriMarket . . . . .   | 10 |
| 4.1 | Récapitulatif des technologies utilisées . . . . .         | 15 |
| 4.2 | État de livraison des fonctionnalités . . . . .            | 17 |
| 4.3 | Principaux endpoints de l'API AfriMarket . . . . .         | 18 |
| B.1 | Entités de la base de données AfriMarket . . . . .         | 24 |

## Glossaire

---

|                           |                                                                                                                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Marketplace</b>        | Plateforme numérique mettant en relation des offreurs et des demandeurs de biens ou de services sans que la plateforme ne soit elle-même propriétaire des biens.                       |
| <b>Mobile Money</b>       | Service financier permettant d'effectuer des paiements, transferts et retraits d'argent via un téléphone mobile, très répandu en Afrique subsaharienne (Orange Money, MTN MoMo, etc.). |
| <b>Point relais</b>       | Lieu physique tiers (boutique, dépôt) servant d'intermédiaire de livraison entre le producteur et le consommateur, évitant ainsi la livraison à domicile.                              |
| <b>Offre groupée</b>      | Mécanisme par lequel un producteur ne lance la production ou la livraison que si la quantité minimale commandée est atteinte par l'ensemble des acheteurs.                             |
| <b>Wallet virtuel</b>     | Portefeuille électronique interne à l'application, crédité lors du paiement d'une commande et débitable vers le compte Mobile Money du producteur.                                     |
| <b>Token JWT</b>          | Jeton numérique signé permettant d'authentifier un utilisateur auprès de l'API sans stocker d'état côté serveur.                                                                       |
| <b>Expo / Expo Router</b> | Framework JavaScript facilitant le développement d'applications React Native multi-plateformes (iOS, Android, Web) avec un système de routage par fichiers.                            |
| <b>Spring Boot</b>        | Framework Java permettant de créer des applications web autonomes avec un minimum de configuration.                                                                                    |

# Table des matières

---

|                                                             |            |
|-------------------------------------------------------------|------------|
| <b>Remerciements</b>                                        | <b>i</b>   |
| <b>Résumé</b>                                               | <b>ii</b>  |
| <b>Abstract</b>                                             | <b>iii</b> |
| <b>Liste des figures</b>                                    | <b>iv</b>  |
| <b>Liste des tableaux</b>                                   | <b>v</b>   |
| <b>Liste des Abréviations</b>                               | <b>v</b>   |
| <b>Glossaire</b>                                            | <b>vi</b>  |
| <b>1 Introduction générale</b>                              | <b>2</b>   |
| <b>2 Concepts et état de l'art</b>                          | <b>4</b>   |
| 2.1 Concepts fondamentaux . . . . .                         | 4          |
| 2.1.1 La marketplace pair-à-pair (P2P) . . . . .            | 4          |
| 2.1.2 Les offres groupées (group buying) . . . . .          | 4          |
| 2.1.3 Le Mobile Money en Afrique centrale . . . . .         | 4          |
| 2.1.4 Le réseau de points relais . . . . .                  | 5          |
| 2.1.5 L'architecture RESTful et les microservices . . . . . | 5          |
| 2.1.6 La sécurité par JSON Web Token . . . . .              | 5          |
| 2.2 État de l'art des solutions existantes . . . . .        | 6          |
| 2.2.1 Plateformes agricoles africaines . . . . .            | 6          |
| 2.2.2 Frameworks de développement mobile . . . . .          | 6          |
| 2.2.3 Frameworks backend pour les API mobiles . . . . .     | 6          |
| 2.3 Bilan . . . . .                                         | 7          |
| <b>3 Méthodologie</b>                                       | <b>8</b>   |
| 3.1 Analyse des besoins . . . . .                           | 8          |
| 3.1.1 Besoins fonctionnels . . . . .                        | 8          |
| 3.1.1.1 Consommateur . . . . .                              | 8          |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 3.1.1.2  | Producteur . . . . .                                      | 8         |
| 3.1.1.3  | Gestionnaire de point relais . . . . .                    | 8         |
| 3.1.1.4  | Administrateur . . . . .                                  | 8         |
| 3.1.1.5  | Finance . . . . .                                         | 9         |
| 3.1.2    | Besoins non fonctionnels . . . . .                        | 9         |
| 3.2      | Conception . . . . .                                      | 10        |
| 3.2.1    | Modèle conceptuel des données . . . . .                   | 10        |
| 3.2.2    | Diagramme des cas d'utilisation . . . . .                 | 11        |
| 3.2.3    | Cycle de vie d'une offre et d'une commande . . . . .      | 12        |
| 3.3      | Architecture du système . . . . .                         | 12        |
| 3.3.1    | Architecture globale . . . . .                            | 12        |
| 3.3.2    | Organisation des applications mobiles . . . . .           | 13        |
| 3.4      | Bilan méthodologique . . . . .                            | 13        |
| <b>4</b> | <b>Implémentation et résultats</b>                        | <b>14</b> |
| 4.1      | Choix technologiques . . . . .                            | 14        |
| 4.1.1    | Frontend mobile : React Native avec Expo . . . . .        | 14        |
| 4.1.2    | Gestion d'état : Zustand . . . . .                        | 14        |
| 4.1.3    | Backend : Spring Boot 4 avec Java 17 . . . . .            | 14        |
| 4.1.4    | Administration web : Spring MVC avec Thymeleaf . . . . .  | 15        |
| 4.2      | Présentation de l'application AfriMarket . . . . .        | 15        |
| 4.2.1    | Application consommateur . . . . .                        | 15        |
| 4.2.2    | Application producteur . . . . .                          | 16        |
| 4.2.3    | Interface d'administration . . . . .                      | 16        |
| 4.3      | Résultats . . . . .                                       | 17        |
| 4.3.1    | Fonctionnalités livrées . . . . .                         | 17        |
| 4.3.2    | Structure de l'API REST . . . . .                         | 17        |
| 4.3.3    | Bilan et perspectives . . . . .                           | 18        |
|          | <b>Conclusion générale</b>                                | <b>19</b> |
|          | <b>A Extraits de code significatifs</b>                   | <b>20</b> |
| A.1      | Filtre d'authentification JWT (backend) . . . . .         | 20        |
| A.2      | Store Wallet avec Zustand (frontend producteur) . . . . . | 21        |
|          | <b>B Modèle de données complet</b>                        | <b>23</b> |
|          | <b>Références bibliographiques</b>                        | <b>25</b> |

## 1. Introduction générale

---

Le développement de l'économie numérique en Afrique subsaharienne connaît une croissance soutenue portée par la progression rapide de la téléphonie mobile et des infrastructures Internet. Dans ce contexte, le secteur agricole, qui représente une part significative du de nombreux pays d'Afrique centrale, reste paradoxalement l'un des moins numérisés. Les agriculteurs manquent d'outils adaptés pour valoriser leur production et rejoindre efficacement les marchés urbains, tandis que les consommateurs souffrent d'une opacité persistante sur la provenance et la qualité des denrées alimentaires.

Ce projet s'inscrit dans cette dynamique de transformation numérique de l'agriculture en proposant AfriMarket, une plateforme mobile pair-à-pair dédiée à la mise en relation directe de producteurs agricoles et de consommateurs au Cameroun. L'idée centrale est d'éliminer les intermédiaires traditionnels souvent à l'origine des marges excessives et des pertes post-récolte, tout en s'appuyant sur des points relais locaux pour gérer la logistique de livraison et sur le Mobile Money pour sécuriser les paiements.

La problématique centrale de ce travail peut se formuler ainsi : comment concevoir et développer une application mobile qui s'adapte aux contraintes spécifiques du marché camerounais - connectivité intermittente, prédominance du paiement mobile, faible culture numérique de certains producteurs - tout en offrant une expérience utilisateur fluide et sécurisée à l'ensemble des acteurs de la chaîne ? Cette question en recouvre plusieurs autres : comment modéliser un système d'offres groupées permettant aux producteurs de ne livrer que lorsqu'un seuil minimal de commandes est atteint ? Comment orchestrer le cycle de vie d'une commande entre les rôles de producteur, de consommateur, de gestionnaire de point relais et d'administrateur ? Comment intégrer un portefeuille virtuel interne tout en prévoyant les passerelles vers les opérateurs de Mobile Money ?

La motivation principale de ce projet naît d'un constat terrain : au marché central de Yaoundé comme dans les marchés de quartier, les agriculteurs des zones péri-urbaines vendent souvent leur production à des prix très inférieurs à ceux affichés en ville, faute de moyens logistiques et de visibilité. AfriMarket vise à corriger cette asymétrie en dotant les producteurs d'un outil de gestion d'offres accessible depuis un simple smartphone et en permettant aux consommateurs de s'organiser collectivement pour atteindre les seuils d'achat. Au-delà de l'enjeu économique, le projet répond à une ambition sociale :

renforcer la souveraineté alimentaire locale et valoriser le travail agricole.

Les objectifs de ce projet sont multiples et structurés en niveaux de priorité. L'objectif principal est de livrer une application mobile fonctionnelle comprenant trois interfaces distinctes — consommateur, producteur et administration — reliées à un backend [REST](#) unifié. Les objectifs secondaires incluent la mise en place d'un système d'offres groupées avec progression visible en temps réel, l'intégration d'un wallet virtuel avec mécanisme de retrait vers Mobile Money, la gestion d'un réseau de points relais et la sécurisation des échanges par authentification [JSON Web Token \(JWT\)](#).

Ce rapport est organisé en trois chapitres principaux. Le premier chapitre — cette introduction — pose le contexte, la problématique, les motivations et les objectifs du projet. Le deuxième chapitre explore les concepts fondamentaux mobilisés et dresse un état de l'art des solutions existantes. Le troisième chapitre détaille la méthodologie suivie : analyse des besoins, conception et architecture. Le quatrième chapitre présente les choix technologiques et les résultats obtenus à travers des captures d'écran et une description des fonctionnalités livrées.

## 2. Concepts et état de l'art

---

### 2.1 Concepts fondamentaux

La compréhension d'AfriMarket nécessite la maîtrise de plusieurs concepts transversaux issus du commerce électronique, de la logistique et du développement mobile.

#### 2.1.1 La marketplace pair-à-pair (P2P)

Une marketplace **Peer-to-Peer (P2P)** est une plateforme numérique qui facilite des transactions directes entre particuliers ou entre petits producteurs et consommateurs, sans que l'opérateur de la plateforme ne soit propriétaire des biens échangés. Contrairement aux places de marché B2C classiques (Business-to-Consumer), le modèle **P2P** repose sur la confiance entre pairs, renforcée par des mécanismes de notation, de vérification d'identité et d'escrow (séquestre des fonds). Airbnb et Uber en sont les exemples emblématiques ; dans le secteur agricole, des plateformes comme Twiga Foods au Kenya ou Hello Tractor en Afrique de l'Ouest illustrent le potentiel de ce modèle pour le continent africain [6].

#### 2.1.2 Les offres groupées (group buying)

Le mécanisme d'offre groupée, popularisé par Groupon à l'international, consiste à conditionner l'activation d'une offre commerciale à l'atteinte d'un seuil minimal de participants. Dans le contexte agricole, ce mécanisme présente un double avantage : il garantit au producteur une taille critique de commande qui justifie le déplacement ou la récolte spécifique, et il permet aux consommateurs de bénéficier de prix de gros tout en achetant en quantité raisonnable. AfriMarket adapte ce concept en affichant un indicateur de progression en temps réel (pourcentage de la quantité minimale atteinte), ce qui crée une dynamique sociale d'entraînement entre les acheteurs.

#### 2.1.3 Le Mobile Money en Afrique centrale

Le Mobile Money désigne un service financier permettant d'effectuer des transactions monétaires via un téléphone mobile, sans nécessiter de compte bancaire traditionnel. En

Afrique subsaharienne, deux opérateurs dominant le marché camerounais : MTN Mobile Money (MoMo) et Orange Money. Avec un taux de bancarisation inférieur à 20% au Cameroun [1], le Mobile Money représente le principal vecteur de paiement numérique pour une large part de la population, y compris les agriculteurs. L’intégration de ces services dans AfriMarket est donc non seulement souhaitable mais indispensable pour l’adoption de la plateforme.

#### **2.1.4 Le réseau de points relais**

Un point relais est un établissement commercial partenaire (kiosque, boutique, dépôt) servant d’intermédiaire entre le producteur et le consommateur pour la remise des commandes. Ce modèle logistique, bien établi en Europe (Mondial Relay, Chronopost), se révèle particulièrement adapté aux villes africaines où l’adressage postal est souvent imprécis et où la livraison à domicile génère des coûts importants. AfriMarket intègre un réseau de points relais géo-référencés, associés à des zones géographiques calquées sur les arrondissements de Yaoundé.

#### **2.1.5 L’architecture RESTful et les microservices**

L’[Application Programming Interface \(API\) REST](#) constitue le style architectural dominant pour les backend d’applications mobiles. Elle repose sur le protocole [Hyper-Text Transfer Protocol \(HTTP\)](#) et organise les ressources autour de verbes sémantiques (GET, POST, PUT, DELETE). Dans AfriMarket, cette architecture permet de découpler complètement les trois interfaces clientes (consommateur, producteur, admin) du backend, offrant une flexibilité maximale pour l’évolution indépendante de chaque composante [4].

#### **2.1.6 La sécurité par JSON Web Token**

Le [JWT](#) est un standard ouvert (RFC 7519) définissant une manière compacte et auto-portée de transmettre des informations entre parties sous forme d’objet [JavaScript Object Notation \(JSON\)](#) signé numériquement. Dans le contexte d’une [API REST](#) sans état (stateless), le [JWT](#) remplace avantageusement la session serveur traditionnelle : le token, émis lors de la connexion et stocké côté client, est joint à chaque requête et vérifié par le serveur sans consultation d’une base de données de session [5].

## 2.2 État de l’art des solutions existantes

### 2.2.1 Plateformes agricoles africaines

Plusieurs initiatives numériques tentent de résoudre le problème de mise en marché des productions agricoles en Afrique. Twiga Foods (Kenya) propose un modèle B2B qui achète les productions directement aux agriculteurs pour les revendre aux épiceries urbaines. Hello Tractor (Nigeria) se concentre sur la location de matériel agricole. Esoko (Ghana) fournit des informations de prix de marché par SMS. En Afrique centrale, les solutions spécifiques au Cameroun restent rares et rarement adaptées aux spécificités linguistiques et logistiques locales.

**Table 2.1** – *Comparaison des plateformes agri-tech existantes*

| Plateforme        | Pays            | Modèle     | Paielement   | Livraison            |
|-------------------|-----------------|------------|--------------|----------------------|
| Twiga Foods       | Kenya           | B2B        | Mobile Money | Propre fleet         |
| Hello Tractor     | Nigeria         | Location   | Carte/Mobile | Non concerné         |
| Esoko             | Ghana           | Info prix  | –            | –                    |
| Jumia Food        | Multi           | B2C resto  | Mobile/Carte | Coursiers            |
| <b>AfriMarket</b> | <b>Cameroun</b> | <b>P2P</b> | <b>MoMo</b>  | <b>Points relais</b> |

### 2.2.2 Frameworks de développement mobile

Le choix d’un framework de développement mobile est crucial car il détermine la maintenance, les performances et la capacité à cibler plusieurs plateformes simultanément. Trois approches coexistent : le développement natif (Swift/Kotlin), le développement hybride (Ionic/Cordova) et le développement cross-platform (Flutter, React Native). React Native, soutenu par Meta, se distingue par son utilisation de JavaScript/TypeScript, sa large communauté et son écosystème Expo qui simplifie considérablement le processus de build et de déploiement [3]. Il s’impose comme le choix naturel dans un contexte académique où la réutilisation des compétences web JavaScript est un avantage.

### 2.2.3 Frameworks backend pour les API mobiles

Spring Boot, maintenu par VMware Pivotal, est le framework Java le plus utilisé en entreprise pour construire des API RESTful. Sa convention-over-configuration, sa gestion intégrée de la sécurité via Spring Security, et son écosystème JPA/Hibernate pour la persistance des données en font un choix robuste et éprouvé. La version 3.x,

basée sur Jakarta EE, apporte des améliorations significatives en termes de performances et de support des architectures réactives [7].

## 2.3 Bilan

L’analyse de l’existant révèle un écosystème agri-tech africain encore fragmenté, dominé par des solutions B2B ou des services d’information incomplets. Aucune plateforme ne propose simultanément les offres groupées, le réseau de points relais géolocalisés, le wallet virtuel et l’interface tripartite (producteur / consommateur / admin) adaptée au marché camerounais. AfriMarket comble ce vide en s’appuyant sur des technologies modernes (React Native avec Expo, Spring Boot 4) et en intégrant nativement les contraintes locales (Mobile Money, gestion par zones, mode hors-ligne partiel). Ce positionnement justifie pleinement le développement d’une solution sur mesure plutôt que l’adaptation d’une solution existante.

## **3. Méthodologie**

---

### **3.1 Analyse des besoins**

#### **3.1.1 Besoins fonctionnels**

Les besoins fonctionnels ont été identifiés en interrogeant les utilisateurs cibles et en analysant les processus actuels de vente agricole. Ils ont été organisés autour des cinq acteurs principaux du système.

##### **3.1.1.1 Consommateur**

Le consommateur doit pouvoir s'inscrire et se connecter via son numéro de téléphone, parcourir et filtrer les offres actives par catégorie (légumes, fruits, tubercules, céréales, élevage) et par zone géographique, visualiser la progression d'une offre groupée, rejoindre une offre en spécifiant la quantité souhaitée, payer via Mobile Money, sélectionner un point relais pour la livraison, confirmer la réception de sa commande et ouvrir un litige en cas de problème.

##### **3.1.1.2 Producteur**

Le producteur doit pouvoir créer une offre avec titre, description, catégorie, unité, prix unitaire, quantité disponible, seuil minimal et date d'expiration, téléverser des photos de sa production, soumettre l'offre à validation administrative, suivre en temps réel les commandes reçues, confirmer l'expédition vers les relais, gérer son portefeuille virtuel et demander un retrait vers son compte Mobile Money.

##### **3.1.1.3 Gestionnaire de point relais**

Le gestionnaire de point relais doit recevoir les notifications de dépôt, signaler la réception des colis et confirmer les remises aux consommateurs.

##### **3.1.1.4 Administrateur**

L'administrateur doit valider ou rejeter les offres et les profils de producteurs, accéder à un tableau de bord avec les indicateurs clés (chiffre d'affaires, commandes, utilisateurs),

gérer les zones, les points relais, les utilisateurs et les paramètres système.

### 3.1.1.5 Finance

L'opérateur finance doit visualiser l'ensemble des transactions, déclencher les virements Mobile Money aux producteurs et gérer les remboursements en cas de litige.

**Table 3.1** – *Matrice des besoins fonctionnels par acteur*

| Acteur       | Besoin fonctionnel principal           | Priorité |
|--------------|----------------------------------------|----------|
| Consommateur | Parcourir et rejoindre des offres      | Haute    |
| Consommateur | Payer via Mobile Money                 | Haute    |
| Consommateur | Confirmer la réception / ouvrir litige | Moyenne  |
| Producteur   | Créer et publier une offre groupée     | Haute    |
| Producteur   | Gérer son wallet virtuel               | Haute    |
| Producteur   | Retirer ses gains vers MoMo            | Haute    |
| Relais       | Confirmer dépôt et remise              | Moyenne  |
| Admin        | Valider/Rejeter les offres             | Haute    |
| Admin        | Tableau de bord analytique             | Moyenne  |
| Finance      | Gérer les transactions et virements    | Haute    |

### 3.1.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes qualitatives que doit respecter le système indépendamment de ses fonctionnalités métier.

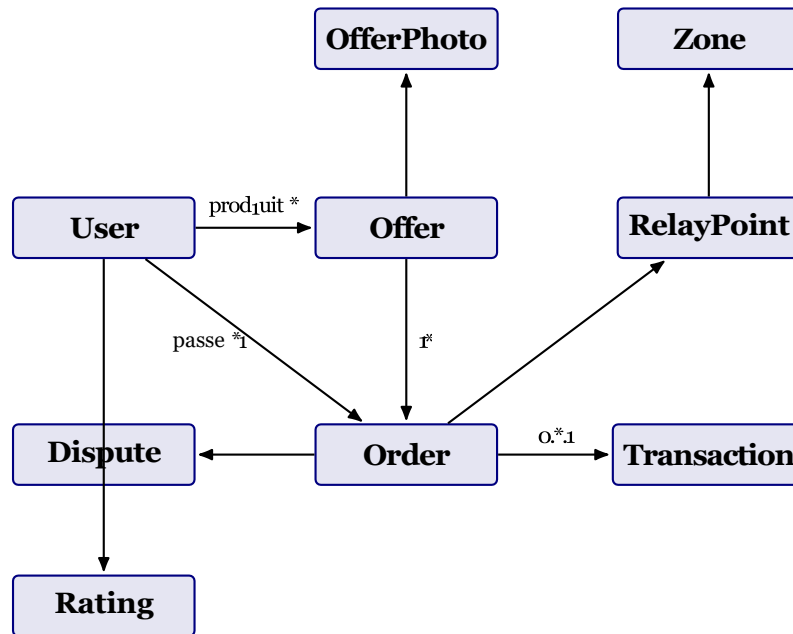
**Table 3.2** – *Besoins non fonctionnels du système AfriMarket*

| Catégorie      | Exigence                                                          | Indicateur de mesure             |
|----------------|-------------------------------------------------------------------|----------------------------------|
| Performance    | Temps de réponse de l'API inférieur à 500 ms                      | Mesure avec Postman / JMeter     |
| Disponibilité  | Uptime du backend supérieur à 99%                                 | Monitoring uptime                |
| Sécurité       | Authentification par <a href="#">JWT</a>                          | Rotation du token toutes les 24h |
| Sécurité       | Chiffrement des mots de passe (BCrypt)                            | Audit du code SecurityConfig     |
| Scalabilité    | Architecture stateless permettant la mise à l'échelle horizontale | Containerisation Docker          |
| Utilisabilité  | Interface utilisable sans formation préalable                     | Tests utilisateurs               |
| Maintenabilité | Code couvert à 60% par des tests unitaires                        | Rapport JaCoCo                   |
| Compatibilité  | Support Android 8+ et iOS 14+                                     | Tests sur émulateurs             |
| Hors-ligne     | Affichage des offres récentes sans connexion                      | Mode dégradé avec cache local    |

## 3.2 Conception

### 3.2.1 Modèle conceptuel des données

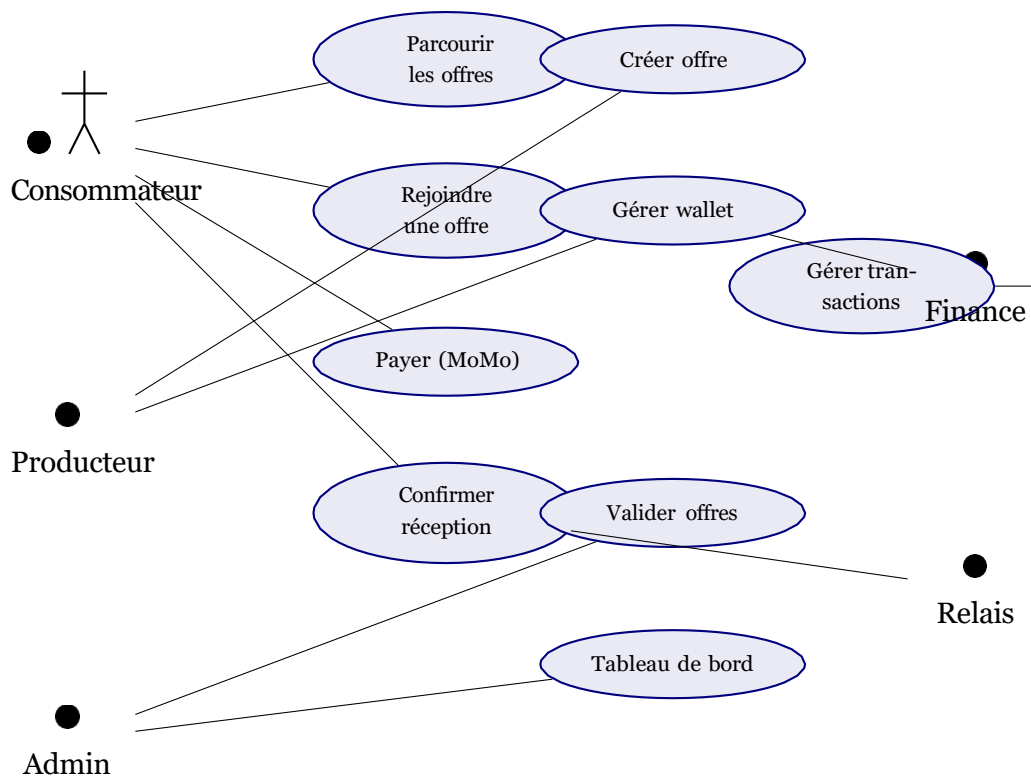
Le modèle de données d'AfriMarket s'articule autour de neuf entités principales. L'entité User centralise les informations de tous les acteurs (consommateurs, producteurs, gestionnaires de relais, administrateurs, agents financiers) grâce à un champ role de type énuméré. L'entité Offer modélise une offre groupée avec ses seuils, son statut et ses photos associées. L'entité Order représente la participation d'un consommateur à une offre, avec capture du prix au moment de l'achat. Les entités Transaction et Wallet (virtuel) gèrent les flux financiers internes. L'entité RelayPoint regroupe les points relais géoréférencés associés à des Zone. Enfin, les entités Rating et Dispute couvrent les mécanismes de réputation et de résolution des litiges.



**Figure 3.1** – Diagramme entité-association simplifié d'AfriMarket

### 3.2.2 Diagramme des cas d'utilisation

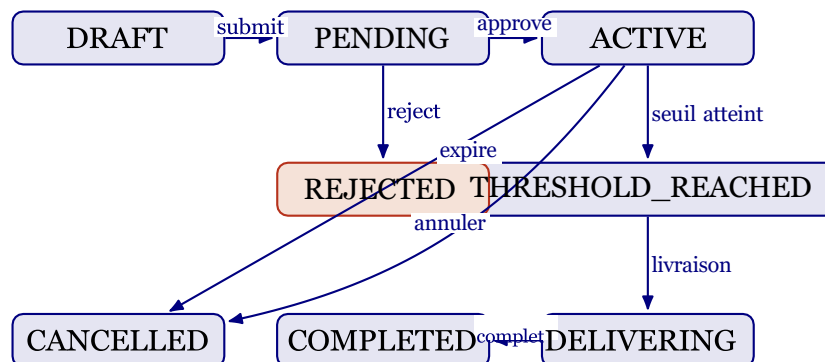
Le diagramme de cas d'utilisation ci-dessous représente les interactions entre les cinq acteurs principaux et les grandes fonctions du système.



**Figure 3.2** – Diagramme des cas d'utilisation d'AfriMarket

### 3.2.3 Cycle de vie d'une offre et d'une commande

Le cycle de vie d'une offre dans AfriMarket suit une séquence d'états précisément définis. Une offre débute à l'état *DRAFT* lors de sa création par le producteur, puis passe à *PENDING* après sa soumission à validation. L'administrateur la valide (*ACTIVE*) ou la rejette (*REJECTED*). Une offre active collecte des commandes jusqu'à atteindre son seuil minimal (*THRESHOLD\_REACHED*), après quoi la livraison est confirmée (*DELIVERING*), puis l'offre est clôturée (*COMPLETED*) ou annulée (*CANCELLED*) si le seuil n'est pas atteint avant expiration.



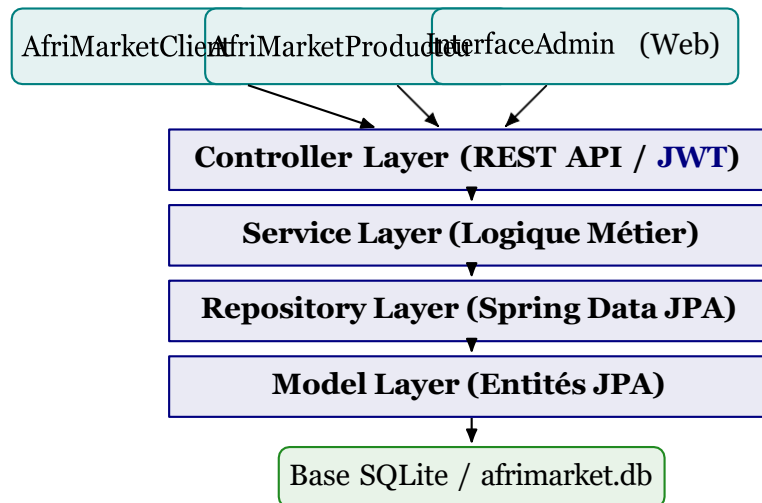
**Figure 3.3** – Cycle de vie d'une offre dans AfriMarket

## 3.3 Architecture du système

L'architecture d'AfriMarket suit un schéma en couches (layered architecture) côté backend et un pattern *Model-View-ViewModel (MVVM)* côté applications mobiles.

### 3.3.1 Architecture globale

Le backend Spring Boot est organisé en quatre couches distinctes : la couche *Controller* expose les endpoints *REST* et prend en charge la validation des entrées ; la couche *Service* contient la logique métier ; la couche *Repository* abstrait l'accès aux données via Spring Data *Jakarta Persistence API (JPA)* ; enfin, la couche *Model* définit les entités persistées dans la base de données SQLite via l'*Object-Relational Mapping (ORM)* Hibernate.



**Figure 3.4** – *Architecture en couches d'AfriMarket*

### 3.3.2 Organisation des applications mobiles

Les deux applications mobiles (AfriMarket Client et AfriMarket Producteur) suivent la même structure dictée par Expo Router. Les routes sont organisées par fichiers dans le dossier app/, avec des groupes de routes pour l'authentification ((auth)) et les onglets principaux ((tabs)). La couche services/ encapsule les appels [API](#) vers le backend. La couche store/ gère l'état global avec Zustand, bibliothèque de gestion d'état légère et sans boilerplate. La couche components/ organise les composants réutilisables, eux-mêmes subdivisés par domaine fonctionnel.

## 3.4 Bilan méthodologique

La phase d'analyse et de conception a permis d'établir des fondations solides pour le développement. L'identification claire des cinq rôles utilisateurs et de leurs besoins spécifiques a guidé la définition des entités du domaine et l'organisation des routes [API](#). Le choix d'une architecture en couches standard, combiné à une séparation nette entre les applications mobiles et le backend via une [API REST](#), garantit une bonne maintenabilité et ouvre la voie à des évolutions futures, comme l'ajout d'un service de notifications push ou l'intégration d'une [API](#) de cartographie pour la géolocalisation des points relais. La décision de reporter la migration vers une base de données de production (PostgreSQL) à une phase ultérieure, en utilisant SQLite pour le développement, a permis d'accélérer les cycles d'itération.

## 4. Implémentation et résultats

---

### 4.1 Choix technologiques

#### 4.1.1 Frontend mobile : React Native avec Expo

React Native a été retenu pour le développement des applications mobiles AfriMarket Client et AfriMarket Producteur. Ce framework JavaScript, développé par Meta, permet de produire des applications natives pour iOS et Android à partir d'une base de code unique, réduisant ainsi considérablement les délais de développement. Expo, surcouche qui simplifie la gestion du cycle de vie React Native, apporte le routeur par fichiers (Expo Router), la gestion simplifiée des assets et des outils de débogage efficaces. TypeScript a été adopté comme langage de programmation pour bénéficier du typage statique, réduisant les erreurs à l'exécution et améliorant l'expérience de développement avec l'autocomplétion [2].

#### 4.1.2 Gestion d'état : Zustand

Zustand a été préféré à Redux pour la gestion d'état global dans les applications mobiles. Sa philosophie minimaliste (absence de reducers, d'actions et de dispatchers) réduit le code boilerplate tout en offrant des performances équivalentes. Les stores (auth.store.ts, offer.store.ts, wallet.store.ts) encapsulent chacun l'état et les actions d'un domaine fonctionnel.

#### 4.1.3 Backend : Spring Boot 4 avec Java 17

Spring Boot 4, basé sur Jakarta EE 10, constitue le socle du backend. Spring Security gère l'authentification et les autorisations : un filtre [JWT](#) personnalisé (JwtAuthFilter) intercepte chaque requête, valide le token et injecte l'utilisateur dans le contexte de sécurité. Spring Data [JPA](#) avec Hibernate assure la persistance objet-relationnelle. Lombok élimine le code répétitif (getters/setters/constructeurs). La base de données SQLite, via le driver Xerial, est utilisée pour le développement ; son fichier afrimarket.db constitue un artefact deployable autonome.

#### 4.1.4 Administration web : Spring MVC avec Thymeleaf

L'interface d'administration est une application web classique rendue côté serveur via le moteur de templates Thymeleaf, intégrée au même serveur Spring Boot. Cette approche simplifie le déploiement (un seul artefact) et exploite Spring Security pour sécuriser l'accès à l'espace admin par une authentification formulaire distincte de l'authentification [JWT](#) utilisée par les applications mobiles.

**Table 4.1** – Récapitulatif des technologies utilisées

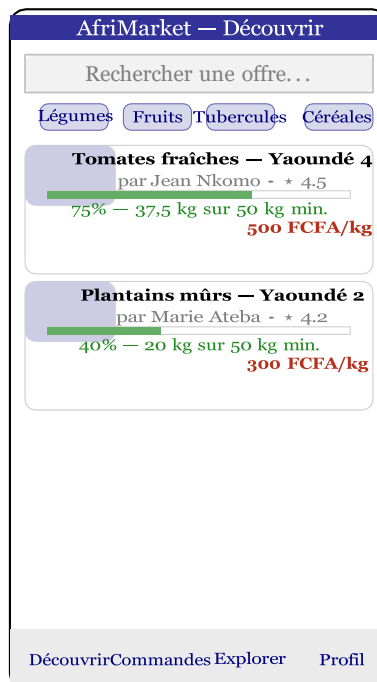
| Composante                   | Technologie                           | Version |
|------------------------------|---------------------------------------|---------|
| Application consommateur     | React Native / Expo                   | SDK 53  |
| Application producteur       | React Native / Expo                   | SDK 53  |
| Langage frontend             | TypeScript                            | 5.x     |
| Gestion d'état               | Zustand                               | 4.x     |
| Backend <a href="#">API</a>  | Spring Boot                           | 4.0.6   |
| Langage backend              | Java                                  | 17      |
| Sécurité                     | Spring Security + <a href="#">JWT</a> | 6.x     |
| Persistance                  | Spring Data JPA / Hibernate           | 6.x     |
| Base de données              | SQLite                                | 3.x     |
| Templates admin              | Thymeleaf                             | 3.x     |
| Build backend                | Maven                                 | 3.9     |
| Gestion dépendances frontend | npm                                   | 10.x    |

## 4.2 Présentation de l'application AfriMarket

### 4.2.1 Application consommateur

L'application consommateur (AfriMarket Client) propose cinq écrans principaux accessibles via une barre de navigation inférieure. L'écran *Découvrir* affiche les offres actives filtrables par catégorie et par zone, avec un champ de recherche textuelle. Chaque carte d'offre présente le titre, la catégorie, le prix unitaire, le producteur et une barre de progression indiquant le pourcentage du seuil minimal atteint. En appuyant sur une offre, l'utilisateur accède à l'écran de détail qui récapitule toutes les informations, liste les points relais disponibles dans la zone et permet de spécifier la quantité souhaitée avant de rejoindre l'offre. L'écran *Commandes* liste l'historique des participations avec

leur statut. L'écran *Explorer* permet de parcourir les zones géographiques. Un écran de profil gère les informations personnelles.



**Figure 4.1** – Maquette de l'écran « Découvrir » de l'application consommateur

#### 4.2.2 Application producteur

L'application producteur (AfriMarket Producteur) cible les agriculteurs et leur offre un tableau de bord synthétique affichant les revenus du mois, le nombre d'offres actives et le solde du wallet disponible. L'écran de création d'offre guide l'utilisateur à travers un formulaire multi-étapes : informations générales (titre, description, catégorie), tarification et quantités (prix unitaire, quantité disponible, seuil minimal, quantité minimale par acheteur), photos de la production, et dates (disponibilité et expiration). Une fois soumise, l'offre entre dans la file de validation administrative. L'écran de liste des offres affiche le statut de chaque offre et, pour les offres actives, la liste des commandes reçues avec les coordonnées des acheteurs. L'écran Wallet permet au producteur de visualiser son solde disponible, le montant en attente (commandes confirmées non encore virées) et l'historique des transactions, et de déclencher une demande de retrait vers son compte Mobile Money (minimum 500 [Franc CFA \(FCFA\)](#)).

#### 4.2.3 Interface d'administration

L'interface d'administration, accessible via navigateur, est protégée par une authentification par formulaire. Le tableau de bord principal affiche des indicateurs clés : chiffre d'affaires total, nombre de commandes actives, utilisateurs enregistrés et offres en

attente de validation. Des vues dédiées permettent de gérer les utilisateurs (liste, détail, activation/désactivation), les offres (validation, rejet avec motif, consultation des photos), les commandes (suivi du cycle de vie), les transactions financières, les points relais (création, modification, association à une zone) et les paramètres système. L'ensemble est rendu via des templates Thymeleaf avec une mise en forme CSS administrative sobre et fonctionnelle.

## 4.3 Résultats

### 4.3.1 Fonctionnalités livrées

L'ensemble des fonctionnalités prioritaires identifiées lors de la phase d'analyse a été implémenté. Le tableau suivant synthétise l'état de livraison :

**Table 4.2** – *État de livraison des fonctionnalités*

| Fonctionnalité                  | Priorité | Statut     |
|---------------------------------|----------|------------|
| Inscription / Connexion (JWT)   | Haute    | Livré      |
| Parcourir et filtrer les offres | Haute    | Livré      |
| Rejoindre une offre groupée     | Haute    | Livré      |
| Progression en temps réel       | Haute    | Livré      |
| Paieement Mobile Money          | Haute    | Simulation |
| Créer et soumettre une offre    | Haute    | Livré      |
| Téléversement de photos         | Moyenne  | Livré      |
| Validation admin des offres     | Haute    | Livré      |
| Wallet virtuel et retrait       | Haute    | Livré      |
| Tableau de bord admin           | Moyenne  | Livré      |
| Gestion des points relais       | Moyenne  | Livré      |
| Notifications push              | Faible   | Non livré  |
| Géolocalisation des relais      | Faible   | Non livré  |

### 4.3.2 Structure de l'API REST

L'API REST expose des ressources organisées selon les conventions du standard. Les endpoints sont regroupés sous le préfixe `/api/v1/` et protégés par l'authentification JWT, à l'exception des routes d'authentification (`/api/v1/auth/login`, `/api/v1/auth/register`).

**Table 4.3** – Principaux endpoints de l'API AfriMarket

| Méthode | Endpoint                   | Description                           |
|---------|----------------------------|---------------------------------------|
| POST    | /api/v1/auth/login         | Authentification, retourne un JWT     |
| POST    | /api/v1/auth/register      | Inscription d'un nouvel utilisateur   |
| GET     | /api/v1/offers             | Liste des offres actives (filtrables) |
| GET     | /api/v1/offers/{id}        | Détail d'une offre                    |
| POST    | /api/v1/offers/{id}/join   | Rejoindre une offre groupée           |
| GET     | /api/v1/offers/mine        | Mes offres (producteur)               |
| POST    | /api/v1/offers             | Créer une offre (producteur)          |
| POST    | /api/v1/offers/{id}/submit | Soumettre une offre à validation      |
| GET     | /api/v1/orders/mine        | Mes commandes (consommateur)          |
| GET     | /api/v1/wallet             | Solde et infos du wallet              |
| POST    | /api/v1/wallet/withdraw    | Demander un retrait MoMo              |
| GET     | /api/v1/zones              | Liste des zones géographiques         |
| GET     | /api/v1/relays             | Liste des points relais               |

### 4.3.3 Bilan et perspectives

Le projet AfriMarket a atteint ses objectifs principaux en livrant une solution mobile complète et fonctionnelle répondant aux besoins identifiés des producteurs et consommateurs camerounais. Les trois interfaces (consommateur, producteur, admin) s'articulent correctement autour du backend unifié, et le mécanisme d'offres groupées constitue la valeur ajoutée principale du système.

Plusieurs améliorations sont envisagées pour les versions futures. L'intégration réelle des API Mobile Money (MTN et Orange) remplacerait la simulation actuelle et permettrait un déploiement en production. L'ajout des notifications push (via Firebase Cloud Messaging) améliorerait l'engagement des utilisateurs. La migration de SQLite vers PostgreSQL garantirait la fiabilité et les performances en environnement de production. Enfin, l'implémentation d'un système de recommandation basé sur les préférences d'achat et la géolocalisation enrichirait l'expérience de découverte des offres.

## Conclusion générale

---

Ce rapport a présenté le projet AfriMarket, une application mobile marketplace agricole pair-à-pair développée pour le contexte camerounais. Partant d'un problème de terrain bien identifié — la difficulté pour les agriculteurs péri-urbains d'accéder aux marchés urbains à prix équitable — nous avons conduit un processus complet allant de l'analyse des besoins à la livraison d'une solution fonctionnelle.

L'état de l'art a confirmé l'absence de solution adaptée au marché camerounais combinant offres groupées, réseau de points relais, wallet virtuel et prise en charge native du Mobile Money. Ce vide justifiait le développement d'AfriMarket comme solution sur mesure plutôt que l'adaptation d'une plateforme existante.

La phase de conception a permis d'établir un modèle de données robuste autour de neuf entités interconnectées, un découpage clair des rôles utilisateurs et une architecture en couches garantissant la maintenabilité. Le choix de React Native avec Expo pour le frontend et de Spring Boot 4 pour le backend a été motivé par la productivité, la maturité des écosystèmes et l'adéquation au contexte mobile africain.

Les résultats obtenus attestent de la faisabilité technique de la solution : les fonctionnalités prioritaires sont livrées et les architectures retenues tiennent leurs promesses en termes de séparation des préoccupations et d'évolutivité. L'intégration réelle des paiements Mobile Money et le déploiement sur les stores iOS et Android constituent les prochaines étapes naturelles pour transformer ce prototype académique en produit commercialisable.

Au-delà de l'aspect technique, ce projet a été une expérience enrichissante de gestion de la complexité d'un système multi-acteurs, de prise de décision architecturale sous contrainte de temps, et d'adaptation des meilleures pratiques du développement logiciel aux réalités du terrain africain.

## A. Extraits de code significatifs

---

### A.1 Filtre d'authentification JWT (backend)

**Listing A.1** – *JwtAuthFilter.java – Filtre Spring Security*

```
1 // Filtre Spring Security : intercepte chaque requete HTTP ,
2 // extrait le token JWT de l'en-tete Authorization ,
3 // le valide et injecte l'utilisateur dans le contexte de
  securite.
4 @Component
5 @RequiredArgsConstructor
6 public class JwtAuthFilter extends OncePerRequestFilter {
7
8     private final JwtService jwtService ;
9     private final UserDetailsService userDetailsService ;
10
11     @Override
12     protected void doFilterInternal (HttpServletRequest request ,
13                                     HttpServletResponse response , FilterChain
14                                     filterChain )
15         throws ServletException , IOException {
16
17         final String authHeader = request.getHeader("
18             Authorization ");
19
20         // Si l'en-tete est absent ou ne commence pas par "
21         Bearer ", on passe
22         if (authHeader == null || !authHeader.startsWith("
23             Bearer_")) {
24             filterChain . doFilter( request , response );
25             return ;
26         }
27
28         // Extraction du token (on retire le prefixe "Bearer ")
```

```

25     final String jwt = authHeader.substring(7);
26     final String userPhone = jwtService.extractUsername(jwt
27         );
28     // Si le token est valide et qu'aucune authentication
29     n'est en cours
30     if (userPhone != null &&
31         SecurityContextHolder.getContext().
32         getAuthentication() == null) {
33
34         UserDetails userDetails =
35             userDetailsService.loadUserByUsername(userPhone
36                 );
37
38         if (jwtService.isTokenValid(jwt, userDetails)) {
39             // Injection de l'utilisateur dans le contexte
40             de securite
41             UsernamePasswordAuthenticationToken authToken =
42                 new UsernamePasswordAuthenticationToken (
43                 userDetails , null ,
44                 userDetails.getAuthorities());
45             authToken.setDetails (
46                 new WebAuthenticationDetailsSource ()
47                 .buildDetails (request));
48             SecurityContextHolder.getContext ()
49                 .setAuthentication (authToken);
50         }
51     }
52     filterChain.doFilter( request , response );
53 }

```

## A.2 Store Wallet avec Zustand (frontend producteur)

**Listing A.2** – *wallet.store.ts* – Gestion d'état avec Zustand

```

1 // Store Zustand pour le wallet du producteur.
2 // Encapsule l'état (wallet, transactions, loading) et les
  actions.

```

```
3 interface WalletState {
4   wallet: WalletDto | null;
5   transactions: TransactionDto [];
6   loading: boolean;
7   fetchWallet: () => Promise <void>;
8   fetchTransactions: () => Promise <void>;
9 }
10
11 export const useWalletStore = create <WalletState>((set) => ({
12   wallet: null,
13   transactions: [],
14   loading: false,
15
16   // Charge le solde du wallet depuis l'API backend
17   fetchWallet: async () => {
18     set({ loading: true });
19     try {
20       const data = await apiGet <WalletDto>(API.WALLET);
21       set({ wallet: data });
22     } finally {
23       set({ loading: false });
24     }
25   },
26
27   // Charge l'historique des transactions
28   fetchTransactions: async () => {
29     const data = await apiGet <TransactionDto []>(API.
30       TRANSACTIONS);
31     set({ transactions: data });
32   },
33 }));
```

## B. Modèle de données complet

### Annexe B – Modèle de données AfriMarket

**Table B.1** – *Entités de la base de données AfriMarket*

| Entité      | Champs principaux                                                                                                                   | Relations                            |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| User        | id, phone, fullName, role, status, momoNumber, momoProvider, zone, ratingAvg                                                        | ManyToOne Zone                       |
| Offer       | id, producer, title, category, unit, pricePerUnit, availableQty, minThreshold, minQtyPerBuyer, currentQtyOrdered, status, expiresAt | ManyToOne User, OneToMany OfferPhoto |
| Order       | id, offer, buyer, relay, qtyOrdered, unitPriceSnapshot, totalAmount, status, paymentProvider, paidAt                                | ManyToOne Offer, User, RelayPoint    |
| Transaction | id, order, type, amount, commission, netAmount, momoRef, status                                                                     | ManyToOne Order                      |
| RelayPoint  | id, name, address, zone, latitude, longitude                                                                                        | ManyToOne Zone                       |
| Zone        | id, name, region                                                                                                                    | –                                    |
| OfferPhoto  | id, offer, url, isPrimary                                                                                                           | ManyToOne Offer                      |
| Dispute     | id, order, reason, status, resolution                                                                                               | ManyToOne Order                      |
| Rating      | id, rater, rated, offer, score, comment                                                                                             | ManyToOne User, Offer                |

## Références bibliographiques

---

- [1] BANQUE MONDIALE. *Cameroun — Vue d'ensemble économique 2023*. <https://www.banquemondiale.org/fr/country/cameroon/overview>. Consulté en 2025. 2023.
- [2] Boris CHERNY. *Programming TypeScript : Making Your JavaScript Applications Scale*. Sebastopol, CA : O'Reilly Media, 2019. ISBN : 978-1492037651.
- [3] Bonnie EISENMAN. *Learning React Native : Building Native Mobile Apps with JavaScript*. Sebastopol, CA : O'Reilly Media, 2015. ISBN : 978-1491929001.
- [4] Roy Thomas FIELDING. « Architectural Styles and the Design of Network-based Software Architectures ». Thèse de doct. University of California, Irvine, 2000. URL : <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [5] Michael B. JONES, John BRADLEY et Nat SAKIMURA. « JSON Web Token (JWT) ». In : *IETF RFC 7519* (2015). URL : <https://tools.ietf.org/html/rfc7519>.
- [6] TWIGA FOODS. *Twiga Foods — Connecting African Food Systems*. <https://twiga.com>. Consulté en 2025. 2023.
- [7] VMWARE PIVOTAL. *Spring Boot Reference Documentation 3.x*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Consulté en 2025. 2024.